Day 1 – End-to-End ML Pipeline & ML-as-a-Service · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 1 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Establish the platform's ML backbone by shipping a working, persisted churn model exposed as a simple ML service. This gives the business an immediate baseline to flag at-risk customers from day one.

Dataset

Telco Customer Churn – Kaggle: blastchar/telco-customer-churn

Prerequisites

Use only this day's libraries: pandas, scikit-learn, joblib. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Stand up the RetailMind repo and build an end-to-end churn-prediction pipeline (ingest → train → evaluate → persist) as the platform's first service.

What we'll do in this iteration — steps & hints (no code):

1. Initialise the RetailMind repo with a clean structure (data/, src/, models/, notebooks/) and a README stating the product vision.
2. Load the Telco Churn CSV with pandas; inspect the schema, target balance and missing values.
3. Build one end-to-end pipeline: split → preprocess (encode categoricals + scale) → train a baseline classifier → evaluate (accuracy, precision/recall, ROC-AUC).
4. Persist the fitted pipeline with joblib and wrap a thin predict() helper as the platform's first ML service.
5. Record the baseline metrics in the README — every later day measures improvement against this number.

Try-It-Out Challenge

Next iteration: Tomorrow you swap the single baseline for stronger algorithms (Decision Tree, Random Forest, SVM) and pick a champion.

Day 2 – Decision Tree, Random Forest & SVM · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 2 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Upgrade the churn baseline with tree-based and margin-based algorithms that capture non-linear customer patterns. Higher-recall churn detection lets retention teams reach the right customers earlier.

Dataset

Telco Customer Churn – Kaggle: blastchar/telco-customer-churn

Prerequisites

Use only this day's libraries: scikit-learn. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Train and compare Decision Tree, Random Forest and SVM churn models; select the strongest performer.

What we'll do in this iteration — steps & hints (no code):

1. Reuse Day-1's preprocessing so all models train on identical features.
2. Train three models — Decision Tree, Random Forest, SVM — keeping a fixed random seed for fair comparison.
3. Compare them on the same hold-out set; focus on recall for the churn class (catching at-risk customers matters most).
4. Pick the champion, persist it alongside the Day-1 model, and note why it won.
5. Add a short comparison table to the README so the choice is auditable.

Try-It-Out Challenge

Next iteration: Next you run a systematic multi-model sweep with hyperparameter tuning and a bias/variance diagnosis.

Day 3 – Multi-Model Comparison, Tuning & Bias-Variance · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 3 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Tune and validate competing models to ship a robust, well-generalising churn predictor. Reliable accuracy under real conditions builds business trust in the platform's first decisions.

Dataset

Telco Customer Churn – Kaggle: blastchar/telco-customer-churn

Prerequisites

Use only this day's libraries: scikit-learn (GridSearchCV, cross_val). Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Run a systematic multi-model comparison with hyperparameter tuning; diagnose bias/variance and lock the best model.

What we'll do in this iteration — steps & hints (no code):

1. Define a search grid per candidate model and run GridSearchCV with cross-validation.
2. Compare train vs validation scores to read off bias (underfit) vs variance (overfit).
3. Use learning curves to decide whether more data, more features or more regularisation is needed.
4. Lock the tuned champion and version it (e.g. models/churn_v3.joblib).
5. Document the chosen hyperparameters and CV scores in the README.

Try-It-Out Challenge

Next iteration: Next you engineer features, apply PCA and add boosting/bagging ensembles to finalise the churn engine.

Day 4 – Feature Engineering & Ensembles · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 4 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Boost predictive power with engineered features and ensembles to finalise a production-grade churn model. Better-targeted retention directly protects recurring revenue.

Dataset

Telco Customer Churn – Kaggle: blastchar/telco-customer-churn

Prerequisites

Use only this day's libraries: scikit-learn, XGBoost. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Engineer features, apply PCA, and add boosting/bagging ensembles to maximise churn performance; finalise the model.

What we'll do in this iteration — steps & hints (no code):

1. Engineer domain features (tenure buckets, charge ratios, contract/service flags) and assess their value.
2. Apply PCA or feature selection to cut redundancy without losing signal.
3. Train bagging (Random Forest) and boosting (XGBoost) ensembles; compare against the Day-3 champion.
4. Finalise the production churn model, persist it, and freeze the preprocessing it depends on.
5. Update the README baseline table — this is the Phase-1 deliverable (Milestone Assessment 2).

Try-It-Out Challenge

Next iteration: Phase 2 begins: you introduce deep learning with a Keras ANN and benchmark it against this classical champion.

Day 5 – Neural Networks & ANN (Keras) · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 5 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Introduce deep learning with an ANN propensity model and compare depth against classical ML head-to-head. This proves where neural nets add real value before scaling DL across the product.

Dataset

Telco Customer Churn (apply ANN learned via the MNIST demo) – Kaggle: blastchar/telco-customer-churn; Keras MNIST for the demo

Prerequisites

Use only this day's libraries: TensorFlow / Keras. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build a Keras ANN customer-propensity model and benchmark it against the classical champion from Phase 1.

What we'll do in this iteration — steps & hints (no code):

1. Warm up on the Keras MNIST demo to internalise layers, activations and training loops.
2. Build a small feed-forward ANN for churn: normalised inputs → dense layers → sigmoid output.
3. Train with early stopping; watch validation loss to avoid overfitting the tabular data.
4. Benchmark ANN metrics against the Phase-1 champion and record whether depth actually helped here.
5. Save the model and add an honest 'when to use which' note to the README.

Try-It-Out Challenge

Next iteration: Next you move to images: a CNN that auto-categorises product photos for the catalog.

Day 6 – CNN – Image Classification · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 6 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Add a vision module that classifies product images automatically. Auto-tagging the catalog cuts manual cataloguing effort and powers visual search later.

Dataset

Fashion-MNIST, then Fashion Product Images – Keras datasets (fashion_mnist); Kaggle: paramaggarwal/fashion-product-images-small

Prerequisites

Use only this day's libraries: TensorFlow / Keras. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build a CNN that auto-categorises product images for the catalog.

What we'll do in this iteration — steps & hints (no code):

1. Prototype on Fashion-MNIST to validate your CNN architecture quickly.
2. Move to the real Fashion Product Images: build an image-loading + augmentation pipeline.
3. Design a CNN (conv → pool → dropout → dense) and train it to classify product categories.
4. Evaluate with a confusion matrix to see which categories confuse the model.
5. Save the catalog classifier as the first CV module and log accuracy in the README.

Try-It-Out Challenge

Next iteration: Next you reach production accuracy with transfer learning and add object detection (YOLO).

Day 7 – Transfer Learning & Object Detection · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 7 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Reach production-grade image accuracy with transfer learning and locate products via object detection. This enables automated catalog QA and shelf/product analytics.

Dataset

Fashion Product Images (+ small detection set) – Kaggle: paramaggarwal/fashion-product-images-small; Ultralytics COCO sample for detection

Prerequisites

Use only this day's libraries: TensorFlow / Keras, Ultralytics YOLO. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Use transfer learning (VGG/ResNet) for accurate product classification and detect/locate products in catalog images.

What we'll do in this iteration — steps & hints (no code):

1. Replace the from-scratch CNN with a pretrained backbone (VGG/ResNet); freeze base layers, train a new head.
2. Fine-tune the top layers and compare accuracy against the Day-6 CNN.
3. Run a pretrained YOLO model on catalog images to detect and locate products (bounding boxes).
4. Note where detection helps: catalog QA, multi-product shots, shelf analytics.
5. Package classifier + detector as the upgraded vision module.

Try-It-Out Challenge

Next iteration: Next you segment products (background removal) and add Grad-CAM explainability to close out the CV catalog.

Day 8 – Segmentation & End-to-End CV · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 8 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Deliver clean, background-free catalog images with explainable CV predictions. Consistent imagery lifts conversion and gives the team trust in model decisions.

Dataset

Fashion Product Images – Kaggle: paramaggarwal/fashion-product-images-small

Prerequisites

Use only this day's libraries: TensorFlow / Keras, OpenCV. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Segment products (background removal) for clean catalog images, add Grad-CAM explainability, and package the CV module.

What we'll do in this iteration — steps & hints (no code):

1. Apply a segmentation approach (U-Net style or a pretrained segmenter) to mask the product and remove the background.
2. Use OpenCV to composite clean catalog images on a uniform background.
3. Add Grad-CAM heatmaps so each classification decision is explainable to the catalog team.
4. Package classification + detection + segmentation + explainability into one CV module with a simple interface.
5. This closes Phase 2 (Weekly Assessment 03): an explainable computer-vision catalog.

Try-It-Out Challenge

Next iteration: Phase 3 begins: an LSTM that forecasts product demand to drive inventory.

Day 9 – RNN / LSTM – Demand Forecasting · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 9 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Forecast product demand with an LSTM to inform inventory decisions. Accurate demand signals reduce stockouts and overstock, directly cutting cost.

Dataset

Rossmann Store Sales (or Store Item Demand) – Kaggle: Rossmann Store Sales / Store Item Demand Forecasting Challenge

Prerequisites

Use only this day's libraries: TensorFlow / Keras, pandas. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build an LSTM demand-forecasting model for store/product sales.

What we'll do in this iteration — steps & hints (no code):

1. Load and resample the sales data into clean per-store/product time series with pandas.
2. Frame supervised windows (look-back → next-step) and scale the series.
3. Build and train an LSTM to predict next-period demand; hold out the most recent weeks for testing.
4. Evaluate with MAE/RMSE and plot predicted vs actual to sanity-check the fit.
5. Save the forecaster as the demand module feeding inventory planning.

Try-It-Out Challenge

Next iteration: Next you extend to multi-horizon forecasting with a Seq2Seq + attention model.

Day 10 – Seq2Seq & Attention Forecasting · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 10 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Produce multi-horizon demand forecasts with attention for sharper planning. Longer, more accurate forecasts improve procurement and promotion timing.

Dataset

Rossmann Store Sales – Kaggle: Rossmann Store Sales / Store Item Demand Forecasting Challenge

Prerequisites

Use only this day's libraries: TensorFlow / Keras. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Extend to multi-step demand forecasting with attention across multiple horizons.

What we'll do in this iteration — steps & hints (no code):

1. Reshape the data for sequence-to-sequence: encode a history window, decode several future steps.
2. Add an attention mechanism so the decoder can focus on the most relevant past days.
3. Train the Seq2Seq model and compare multi-step accuracy against the Day-9 single-step LSTM.
4. Visualise attention weights to explain which history drives each forecast.
5. Upgrade the demand module to emit multi-week forecasts for procurement.

Try-It-Out Challenge

Next iteration: Next you generate synthetic product images (GAN/VAE) to balance rare catalog categories.

Day 11 – GANs & VAEs – Image Augmentation · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 11 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Generate synthetic product images to balance rare categories and strengthen the vision models. Better-balanced data improves classification fairness and accuracy.

Dataset

Fashion Product Images – Kaggle: paramaggarwal/fashion-product-images-small

Prerequisites

Use only this day's libraries: TensorFlow / Keras. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Use a GAN/VAE to augment images for under-represented product categories.

What we'll do in this iteration — steps & hints (no code):

1. Identify under-represented categories from the Phase-2 confusion matrix.
2. Train a GAN (or VAE) on a thin category to synthesise plausible new product images.
3. Inspect samples for quality and mode-collapse; keep only realistic outputs.
4. Fold the synthetic images back into the Day-6/7 classifier's training set and re-measure per-class accuracy.
5. Document the fairness/accuracy lift (Weekly Assessment 04 milestone).

Try-It-Out Challenge

Next iteration: Next you start text intelligence: clean reviews and a classical sentiment baseline.

Day 12 – NLP & Classical Text Processing · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 12 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Stand up review analytics with a classical sentiment baseline. Turning free-text reviews into signals reveals product issues and customer mood.

Dataset

Amazon product reviews – Hugging Face: amazon_polarity (or Kaggle Amazon reviews)

Prerequisites

Use only this day's libraries: scikit-learn, NLTK. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build the review-processing pipeline: clean text, TF-IDF/embeddings, and a classical sentiment baseline.

What we'll do in this iteration — steps & hints (no code):

1. Load Amazon reviews and build a cleaning pipeline (lowercase, strip noise, tokenise, remove stopwords).
2. Vectorise with TF-IDF (and optionally pre-trained word embeddings).
3. Train a classical sentiment classifier (e.g. logistic regression / linear SVM) as the baseline.
4. Evaluate and surface the most informative terms per sentiment for product insight.
5. Persist the review-processing module — later days upgrade it with BERT.

Try-It-Out Challenge

Next iteration: Next you upgrade text understanding with BERT for sentiment and support-ticket routing.

Day 13 – Transformers & BERT – Text Intelligence · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 13 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Upgrade text understanding with BERT for accurate sentiment and ticket routing. Precise classification speeds support and surfaces product insights.

Dataset

Amazon reviews (+ support-ticket text) – Hugging Face: amazon_polarity; tickets relabeled from reviews

Prerequisites

Use only this day's libraries: Hugging Face Transformers. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Apply/fine-tune BERT for review sentiment and support-ticket classification.

What we'll do in this iteration — steps & hints (no code):

1. Load a pretrained BERT and its tokenizer; encode reviews into model inputs.
2. Fine-tune (or use as a feature extractor) for review sentiment; compare against the Day-12 classical baseline.
3. Reframe a few support-ticket categories and train a ticket-routing classifier on the same backbone.
4. Evaluate per-class and confirm BERT's lift over TF-IDF justifies the cost.
5. Replace the classical sentiment module with the BERT-powered one in the pipeline.

Try-It-Out Challenge

Next iteration: Next you add LLM review summaries and a support chatbot via an LLM API.

Day 14 – LLM Apps – Summaries & Chatbot · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 14 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Summarise review themes and answer customers with an LLM assistant. Instant summaries and 24/7 conversational support cut response time and workload.

Dataset

Amazon reviews (summaries) + product/policy text (bot) – Hugging Face: amazon_polarity; product metadata

Prerequisites

Use only this day's libraries: Hugging Face Transformers, LLM API (OpenAI/Anthropic). Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add LLM review summarization and a support chatbot via an LLM API.

What we'll do in this iteration — steps & hints (no code):

1. Use a summarization pipeline to condense many reviews per product into theme summaries.
2. Wire an LLM API behind a thin client (key from env, never hard-coded).
3. Build a simple support chatbot that keeps conversation history and answers from product/policy text.
4. Add basic prompt templates so answers stay on-brand and grounded in the product metadata.
5. Expose summaries + chatbot as the first assistant module.

Try-It-Out Challenge

Next iteration: Next you generate on-brand marketing imagery with Stable Diffusion.

Day 15 – Diffusion – Marketing Imagery · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 15 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Produce on-brand marketing visuals on demand with diffusion. Instant creative assets reduce design cost and speed up campaigns.

Dataset

None (pretrained Stable Diffusion); prompts from product metadata – Hugging Face Hub: runwayml/stable-diffusion-v1-5

Prerequisites

Use only this day's libraries: Hugging Face diffusers. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Generate on-brand marketing/product imagery with Stable Diffusion.

What we'll do in this iteration — steps & hints (no code):

1. Load a pretrained Stable Diffusion pipeline (GPU runtime) via diffusers.
2. Template prompts from product metadata (category + style + setting) for on-brand visuals.
3. Use negative prompts and guidance scale to control quality and brand consistency.
4. Fix seeds to reproduce approved assets and build a small prompt library.
5. Add the marketing-image generator as a creative module.

Try-It-Out Challenge

Next iteration: Next you create product image variations (img2img) and a short promo video clip.

Day 16 – Image Variations & Video Generation · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 16 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Scale creative output with image variations and short promo videos. Richer, cheaper visual content boosts marketing reach.

Dataset

Catalog images as init (Fashion Product Images); pretrained models – Kaggle: fashion-product-images-small; HF diffusers

Prerequisites

Use only this day's libraries: Hugging Face diffusers. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Create product image variations (img2img) and a short promo video clip.

What we'll do in this iteration — steps & hints (no code):

1. Use img2img with a catalog image as init to produce on-brand variations at controlled strength.
2. Sweep strength/guidance to balance fidelity to the original vs creative reinterpretation.
3. Assemble interpolated frames (or a video pipeline) into a short promo clip.
4. Curate the best variations and clip for a campaign-ready asset set.
5. Extend the creative module with variation + video capabilities.

Try-It-Out Challenge

Next iteration: Next you add TTS voice replies and CLIP-based multimodal product search (end of the generation track).

Day 17 – TTS Voice & Multimodal Search · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 17 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Enable voice support and search-by-image/description with multimodal AI. Natural voice and visual search make the platform more accessible and engaging.

Dataset

Fashion Product Images (CLIP search) + catalog text – Kaggle: fashion-product-images-small

Prerequisites

Use only this day's libraries: Hugging Face TTS, CLIP (transformers). Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add TTS voice replies for support and CLIP-based multimodal (image+text) product search.

What we'll do in this iteration — steps & hints (no code):

1. Add a TTS pipeline so the assistant can speak its support replies.
2. Embed catalog images and text with CLIP into a shared space.
3. Build search-by-text and search-by-image over the CLIP embeddings.
4. Wire voice + multimodal search into the assistant and catalog UX.
5. This completes the generation track (Day-17 milestone): text, image, video, voice & multimodal features live.

Try-It-Out Challenge

Next iteration: Phase 4 begins: build a vector-DB knowledge base for grounded retrieval.

Day 18 – RAG Foundations – Vector DB · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 18 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Create a searchable knowledge base of products and policies via embeddings. Semantic search is the foundation for grounded, accurate assistant answers.

Dataset

Product metadata + store policy/FAQ docs – Kaggle: fashion-product-images-small (styles.csv) + project policy docs

Prerequisites

Use only this day's libraries: sentence-transformers, FAISS / ChromaDB, LangChain. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build a knowledge base from product catalog + policy/FAQ docs; chunk, embed and store in a vector DB.

What we'll do in this iteration — steps & hints (no code):

1. Collect product metadata + policy/FAQ docs into a corpus.
2. Chunk documents sensibly (size + overlap) so retrieval returns coherent passages.
3. Embed chunks with sentence-transformers and store vectors + metadata in FAISS/ChromaDB.
4. Test semantic search with sample queries and inspect the top-k results for relevance.
5. Persist the vector store — it underpins every RAG day that follows.

Try-It-Out Challenge

Next iteration: Next you build the RAG assistant that retrieves and generates grounded answers.

Day 19 – RAG Pipeline – Grounded Q&A · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 19 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Answer customer and staff questions with retrieval-grounded LLM responses. Grounding cuts hallucination and gives trustworthy, source-backed answers.

Dataset

The Day-18 vector store – Project knowledge base (built Day 18)

Prerequisites

Use only this day's libraries: LangChain, LLM API, FAISS / ChromaDB. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build a RAG assistant that retrieves relevant documents and generates grounded answers.

What we'll do in this iteration — steps & hints (no code):

1. Wire retrieval over the Day-18 store into a LangChain RAG chain.
2. Craft a prompt that instructs the LLM to answer only from retrieved context and cite sources.
3. Return answers with their supporting passages so staff can verify.
4. Test on real customer/staff questions; check for hallucination when context is missing.
5. Replace the Day-14 ungrounded chatbot with this grounded RAG assistant.

Try-It-Out Challenge

Next iteration: Next you raise quality with re-ranking, hybrid search and measured evaluation.

Day 20 – Advanced RAG – Re-rank & Evaluate · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 20 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Raise answer quality with re-ranking, hybrid search and measured evaluation. Validated RAG ensures the assistant is reliable enough to ship.

Dataset

Knowledge base + a small Q&A evaluation set – Project knowledge base + curated Q&A pairs

Prerequisites

Use only this day's libraries: LangChain, cross-encoder re-rankers, RAGAS. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add re-ranking and hybrid (keyword + vector) retrieval; evaluate retrieval and answer quality.

What we'll do in this iteration — steps & hints (no code):

1. Add hybrid retrieval (keyword + vector) and merge results.
2. Re-rank candidates with a cross-encoder so the best passages reach the LLM.
3. Curate a small Q&A evaluation set grounded in your knowledge base.
4. Measure faithfulness/answer-relevance with RAGAS and compare against the Day-19 baseline.
5. Lock the retrieval config that scores best and record the numbers.

Try-It-Out Challenge

Next iteration: Next you give the assistant actions: an agent that uses tools for orders and inventory.

Day 21 – AI Agents – Tool Use · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 21 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Let the assistant take actions – order and inventory lookups – through tools. Automating these tasks deflects routine tickets and speeds resolution.

Dataset

Online Retail transactions (orders/inventory) exposed as tools – UCI: Online Retail II

Prerequisites

Use only this day's libraries: LangChain (agents/tools), LLM API. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Build an agent that uses tools to check order status and inventory.

What we'll do in this iteration — steps & hints (no code):

1. Expose order-status and inventory lookups over the Online Retail II data as callable tools.
2. Give the agent clear tool descriptions so it picks the right tool for a request.
3. Add function-calling so the agent fills arguments from the user's question.
4. Test multi-tool questions and handle 'tool returned nothing' gracefully.
5. Connect the agent to the RAG assistant so it can both answer and act.

Try-It-Out Challenge

Next iteration: Next you orchestrate support / order / inventory agents into one workflow.

Day 22 – Multi-Agent Orchestration · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 22 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Coordinate specialised agents into an end-to-end support workflow. Orchestration handles multi-step requests with minimal human involvement.

Dataset

The same tool backends – UCI: Online Retail II + project tools

Prerequisites

Use only this day's libraries: LangGraph, LangChain. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Orchestrate support / order / inventory agents into one workflow.

What we'll do in this iteration — steps & hints (no code):

1. Model the workflow as a graph: a router plus specialised support / order / inventory agents.
2. Define hand-offs and shared state so agents pass context cleanly.
3. Add a supervisor that decides routing and when the task is done.
4. Test a multi-step request that touches more than one agent end-to-end.
5. This reaches the Day-22 milestone: a grounded, action-taking assistant over catalog, policy & orders.

Try-It-Out Challenge

Next iteration: Next you cheaply adapt a small model to the retail domain with LoRA fine-tuning.

Day 23 – Fine-tuning with LoRA · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 23 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Adapt a compact model to the retail domain cheaply with LoRA. A tuned in-house model lowers cost and improves on-brand responses.

Dataset

Support / review text (synthetic support chats) – Hugging Face: amazon_polarity + project support transcripts

Prerequisites

Use only this day's libraries: Hugging Face Transformers + PEFT (LoRA). Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Fine-tune a small open model on retail support conversations with LoRA.

What we'll do in this iteration — steps & hints (no code):

1. Assemble a clean instruction dataset from support chats / relabeled reviews.
2. Pick a small open base model and attach LoRA adapters with PEFT.
3. Fine-tune the adapters (not the full model) to keep cost and memory low.
4. Evaluate tone and accuracy vs the API model; decide where the tuned model replaces it.
5. Save the adapters and document the cost/quality trade-off.

Try-It-Out Challenge

Next iteration: Next you systematise prompts and build an evaluation harness for the assistant.

Day 24 – Prompt Engineering & LLM Eval · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 24 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Engineer and measure prompts to lift assistant quality objectively. Continuous evaluation turns prompt tweaks into measurable gains.

Dataset

Curated prompt/answer evaluation set – Project eval set

Prerequisites

Use only this day's libraries: LangChain, promptfoo / RAGAS. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Systematise prompts and build an evaluation harness for the assistant.

What we'll do in this iteration — steps & hints (no code):

1. Extract prompts into versioned templates instead of inline strings.
2. Build a curated eval set of prompts with expected qualities.
3. Run an evaluation harness (promptfoo/RAGAS) to score variants automatically.
4. A/B competing prompts and keep the winner with evidence.
5. Wire the eval harness so any prompt change is measured before it ships.

Try-It-Out Challenge

Next iteration: Next you add safety: content filters, PII redaction and bias checks.

Day 25 – Responsible AI & Guardrails · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 25 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Make the platform safe and compliant with guardrails, PII handling and bias checks. Responsible AI protects customers and the brand from harm and liability.

Dataset

Platform inputs/outputs + PII test cases – Project logs + synthetic PII cases

Prerequisites

Use only this day's libraries: Guardrails, Microsoft Presidio (PII), fairness checks. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add content-safety filters, PII redaction and bias checks across the AI features.

What we'll do in this iteration — steps & hints (no code):

1. Add input/output guardrails to block unsafe or off-topic content.
2. Redact PII (names, emails, cards) with Presidio before logging or sending to an LLM.
3. Run bias/fairness checks on model decisions across customer segments.
4. Define a fallback/refusal path when guardrails trip.
5. Document the safety posture for compliance review.

Try-It-Out Challenge

Next iteration: Phase 4 (MLOps) deepens: automate the data & feature pipelines feeding every model.

Day 26 – Data & Feature Pipelines · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 26 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Automate reproducible data and feature flows for every model. Reliable pipelines remove manual steps and keep models fed with fresh data.

Dataset

All project datasets – Kaggle / UCI / Hugging Face sources from earlier days

Prerequisites

Use only this day's libraries: pandas, scikit-learn pipelines, (optional) Feast. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Automate ingestion and feature pipelines feeding all models.

What we'll do in this iteration — steps & hints (no code):

1. Inventory every dataset and the features each model consumes.
2. Build reusable, parameterised ingestion + transformation pipelines (no manual notebook steps).
3. Centralise feature definitions so churn, demand and text models share consistent inputs.
4. Add data-validation checks (schema, ranges, nulls) at pipeline boundaries.
5. Schedule the pipelines so models always train on fresh, validated data.

Try-It-Out Challenge

Next iteration: Next you track experiments and register every model with versions.

Day 27 – Experiment Tracking & Registry · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 27 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Govern the model lifecycle with tracked experiments and a registry. Versioned, auditable models make releases safe and reproducible.

Dataset

Training runs from earlier days – Project model artefacts

Prerequisites

Use only this day's libraries: MLflow. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Track experiments and register all models with versions and lifecycle stages.

What we'll do in this iteration — steps & hints (no code):

1. Instrument training runs to log params, metrics and artefacts to MLflow.
2. Register the churn, demand, vision and text models with versions.
3. Assign lifecycle stages (staging/production) and compare runs side-by-side.
4. Backfill the best earlier models into the registry as the current production set.
5. Make the registry the single source of truth for what's deployed.

Try-It-Out Challenge

Next iteration: Next you wrap every model behind one unified inference API.

Day 28 – Model Packaging & APIs · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 28 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Expose every model through a unified inference API. A single clean API turns scattered models into a usable product surface.

Dataset

Persisted models from earlier days – Project model registry

Prerequisites

Use only this day's libraries: FastAPI, pydantic, uvicorn. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Wrap churn, vision, forecasting and the assistant behind one inference API.

What we'll do in this iteration — steps & hints (no code):

1. Load models from the registry behind a FastAPI app.
2. Define typed request/response schemas with pydantic per endpoint (churn, demand, vision, assistant).
3. Add health checks and consistent error handling.
4. Document the API (OpenAPI) so the front-end and partners can integrate.
5. Run it with uvicorn and smoke-test each endpoint.

Try-It-Out Challenge

Next iteration: Next you containerise the services for reproducible deployment.

Day 29 – Containerization · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 29 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Package the platform into portable containers. Reproducible images make deployment consistent across every environment.

Dataset

n/a (application artefacts) – Project codebase

Prerequisites

Use only this day's libraries: Docker, docker-compose. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Containerise the services with reproducible environments.

What we'll do in this iteration — steps & hints (no code):

1. Write a Dockerfile per service with pinned dependencies.
2. Use multi-stage builds to keep images small.
3. Compose the API, model store and supporting services with docker-compose.
4. Externalise config and secrets via environment variables (never bake keys in).
5. Verify the whole stack runs identically from a clean checkout.

Try-It-Out Challenge

Next iteration: Next you add automated tests and a CI/CD pipeline.

Day 30 – CI/CD for ML · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 30 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Automate testing and releases with CI/CD. Every change ships safely and quickly, reducing manual deployment risk.

Dataset

n/a (application artefacts) – Project repository

Prerequisites

Use only this day's libraries: pytest, GitHub Actions. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add automated tests and a CI/CD pipeline for build, test and deploy.

What we'll do in this iteration — steps & hints (no code):

1. Write pytest unit tests for data, model-loading and API endpoints.
2. Add a GitHub Actions workflow: lint → test → build image on every push.
3. Gate merges on green checks so broken code can't ship.
4. Automate image publishing on a tagged release.
5. Document the pipeline so the team can extend it.

Try-It-Out Challenge

Next iteration: Next you take the containerised platform live on the cloud.

Day 31 – Cloud Deployment · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 31 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Take the platform live on the cloud for real users. A hosted deployment turns the project into an accessible, working service.

Dataset

n/a (deployment) – Project containers → cloud

Prerequisites

Use only this day's libraries: Cloud SDK (AWS / GCP / Azure), Docker. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Deploy the containerised platform to the cloud.

What we'll do in this iteration — steps & hints (no code):

1. Choose a cloud target and push images to its container registry.
2. Provision compute (managed container service) and networking for the API.
3. Inject config/secrets securely and run database/store dependencies.
4. Deploy, then verify public endpoints respond correctly.
5. This reaches the Day-31 milestone: a containerised platform deployed live on the cloud.

Try-It-Out Challenge

Next iteration: Next you optimise serving (batching, caching, async) for production load.

Day 32 – Scaling & Serving · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 32 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Make inference fast and scalable for production load. Lower latency and cost keep the platform responsive as usage grows.

Dataset

n/a (serving layer) – Project services

Prerequisites

Use only this day's libraries: FastAPI, Redis (cache), (optional) ONNX. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Optimise serving (batching, caching, async) for real-time and batch workloads.

What we'll do in this iteration — steps & hints (no code):

1. Profile endpoints to find latency and throughput bottlenecks.
2. Add request batching and async handling for high-QPS endpoints.
3. Cache hot results (e.g. repeated lookups) in Redis.
4. Optionally export heavy models to ONNX for faster inference.
5. Load-test and record the latency/cost improvement.

Try-It-Out Challenge

Next iteration: Next you add monitoring, dashboards and drift detection with alerts.

Day 33 – Monitoring & Observability · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 33 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Observe health, performance and drift across the platform. Early drift and error alerts keep the deployed models trustworthy over time.

Dataset

Live inference logs – Project runtime telemetry

Prerequisites

Use only this day's libraries: Prometheus, Grafana, Evidently. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Add logging, metrics dashboards and data/model drift detection with alerts.

What we'll do in this iteration — steps & hints (no code):

1. Emit structured logs and metrics (latency, errors, throughput) from every service.
2. Scrape metrics with Prometheus and build Grafana dashboards.
3. Run Evidently to detect data/model drift against a reference window.
4. Wire alerts so drift or error spikes notify the team.
5. Define what each alert means and the first response.

Try-It-Out Challenge

Next iteration: Next you integrate every module behind one platform UI/API (capstone integration).

Day 34 – Capstone Integration · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 34 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Unify every module into one coherent RetailMind platform. End-to-end integration turns components into a single product the business can use.

Dataset

All project modules – Full RetailMind codebase

Prerequisites

Use only this day's libraries: FastAPI + Streamlit / React. Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Integrate all modules behind one platform UI/API and run end-to-end tests.

What we'll do in this iteration — steps & hints (no code):

1. Bring churn, vision, forecasting, the assistant and creative tools under one API surface.
2. Build a simple UI (Streamlit/React) that exercises each capability.
3. Run end-to-end tests across the full user journey.
4. Fix integration gaps so the modules feel like one product.
5. Tag a release candidate of the unified RetailMind platform.

Try-It-Out Challenge

Next iteration: Final day: production deployment plus retraining schedule, runbook and monitoring handover.

Day 35 – Final Deployment & Maintenance · RetailMind AI Increment

Scenario Context

RetailMind AI is one end-to-end retail AI platform you build incrementally — one module per day, each using exactly the techniques and libraries taught that day. This is the Day 35 increment.

Daily rhythm: 3 teaching sessions → this project increment → Git upload → doubt-clearing. Every increment integrates into the same RetailMind codebase, so by Day 35 you have one real, deployed product.

Objective

Ship the maintained, production RetailMind platform with retraining and a runbook. A documented, self-sustaining system delivers lasting value – not a one-off demo.

Dataset

n/a (production release) – Live RetailMind platform

Prerequisites

Use only this day's libraries: All of the above + scheduler (cron / Airflow). Work inside your RetailMind repo (it persists across days) so this increment integrates with previous ones.

Tasks

Project task: Final production deployment plus a retraining schedule, maintenance runbook and monitoring handover.

What we'll do in this iteration — steps & hints (no code):

1. Do the final production deploy from the Day-34 release candidate.
2. Schedule periodic retraining (cron/Airflow) on fresh data with registry promotion.
3. Write a maintenance runbook: on-call steps, rollback, common incidents.
4. Hand over monitoring dashboards and alert ownership.
5. This is the Final Capstone: a deployed, monitored and maintained RetailMind platform with retraining.

Try-It-Out Challenge

Next iteration: Project complete – RetailMind AI is live, documented and self-sustaining.